

Guia completo de uso do shadcn/ui no Vision Manager

1. Objetivo deste guia

Este documento define **como usar os componentes instalados do shadcn/ui** na plataforma **Vision Manager**, voltada para:

- gestão de clientes;
- envelopes digitais da ótica;
- ordens de serviço;
- vendas;
- acompanhamento de laboratório;
- relatórios e dashboards administrativos.

A ideia é simples: **componentes básicos vêm prontos**, e os componentes de negócio serão criados por cima deles.

2. Princípio da arquitetura visual

2.1. O que o shadcn/ui entrega

O shadcn/ui não funciona como uma biblioteca tradicional instalada e fechada. Ele **copia os componentes para dentro do projeto**, permitindo alterar livremente estilos, variantes e comportamento.

Na prática, os arquivos ficam em algo como:

```
src/components/ui/
```

Exemplos:

```
src/components/ui/button.jsx  
src/components/ui/card.jsx  
src/components/ui/dialog.jsx  
src/components/ui/input.jsx
```

2.2. Regra de ouro do projeto

Não criar do zero:

- botão;
- modal;
- drawer;
- input;
- textarea;
- select;
- checkbox;
- tabela base;
- tabs;
- tooltip;
- toast;
- pagination;
- skeleton.

Criar componentes próprios apenas quando forem de negócio:

- `ClienteResumoCard`;
- `EnvelopeStatusBadge`;
- `OSTimeline`;
- `ReceitaOpticaCard`;
- `PrazoEntregaIndicator`;
- `MaloteResumoCard`;
- `HistoricoClienteTimeline`;
- `VendaResumoCard`.

4. Componentes instalados e como usar

4.1. Button

Quando usar

- salvar;
- cancelar;
- abrir modal;
- editar;
- registrar entrega;
- exportar relatório;
- criar cliente.

Exemplo

```
import { Button } from "@components/ui/button"

export function AcaoPrincipal() {
  return <Button>Novo Cliente</Button>
}
```

Variantes mais úteis

```
<Button>Salvar</Button>
<Button variant="outline">Cancelar</Button>
<Button variant="secondary">Filtrar</Button>
<Button variant="destructive">Excluir</Button>
<Button size="sm">Editar</Button>
```

Padrão recomendado

- Ação principal da tela: **default**
 - Ação secundária: **outline**
 - Ação neutra: **secondary**
 - Ação crítica: **destructive**
-

4.2. Card

Quando usar

- KPI;
- resumo de cliente;
- resumo de OS;
- resumo de vendas;
- bloco de dados da receita.

Exemplo

```
import {
  Card,
  CardContent,
  CardDescription,
  CardHeader,
  CardTitle,
} from "@components/ui/card"

export function CardResumoClientes() {
  return (
    <Card>
      <CardHeader>
```

```

    <CardTitle>Clientes ativos</CardTitle>
    <CardDescription>Total cadastrado no sistema</CardDescription>
  </CardHeader>
  <CardContent>
    <p className="text-3xl font-bold">1.248</p>
  </CardContent>
</Card>
)
}

```

Onde usar no Vision Manager

- Dashboard com indicadores;
- página de cliente;
- detalhes de envelope;
- resumo de pagamento;
- card de próximos retornos.

4.3. Dialog / Modal

Quando usar

- cadastro rápido;
- confirmação não crítica;
- edição de dados curtos;
- visualização de anexo pequeno;
- abertura de formulários rápidos.

Exemplo

```

import { Button } from "@components/ui/button"
import {
  Dialog,
  DialogContent,
  DialogDescription,
  DialogFooter,
  DialogHeader,
  DialogTitle,
  DialogTrigger,
} from "@components/ui/dialog"

export function ModalNovoCliente() {
  return (
    <Dialog>
      <DialogTrigger asChild>
        <Button>Novo Cliente</Button>

```

```

</DialogTrigger>

<DialogContent>
  <DialogHeader>
    <DialogTitle>Cadastrar cliente</DialogTitle>
    <DialogDescription>
      Preencha os dados básicos para iniciar o atendimento.
    </DialogDescription>
  </DialogHeader>

  <div className="grid gap-4 py-4">
    {/* campos do formulário */}
  </div>

  <DialogFooter>
    <Button variant="outline">Cancelar</Button>
    <Button>Salvar</Button>
  </DialogFooter>
</DialogContent>
</Dialog>
)
}

```

Regra importante

Não usar modal para formulários gigantes. Cadastro longo de envelope ou OS deve virar **página própria** ou **drawer amplo**.

4.4. Alert Dialog

Quando usar

Ações irreversíveis ou sensíveis:

- excluir cliente;
- apagar envelope;
- cancelar OS;
- remover pagamento;
- encerrar atendimento.

Exemplo

```

import { Button } from "@components/ui/button"
import {
  AlertDialog,
  AlertDialogAction,

```

```

AlertDialogCancel,
AlertDialogContent,
AlertDialogDescription,
AlertDialogFooter,
AlertDialogHeader,
AlertDialogTitle,
AlertDialogTrigger,
} from "@components/ui/alert-dialog"

export function ConfirmarExclusao() {
  return (
    <AlertDialog>
      <AlertDialogTrigger asChild>
        <Button variant="destructive">Excluir</Button>
      </AlertDialogTrigger>

      <AlertDialogContent>
        <AlertDialogHeader>
          <AlertDialogTitle>Excluir cliente?</AlertDialogTitle>
          <AlertDialogDescription>
            Esta ação não poderá ser desfeita.
          </AlertDialogDescription>
        </AlertDialogHeader>

        <AlertDialogFooter>
          <AlertDialogCancel>Cancelar</AlertDialogCancel>
          <AlertDialogAction>Confirmar exclusão</AlertDialogAction>
        </AlertDialogFooter>
      </AlertDialogContent>
    </AlertDialog>
  )
}

```

4.5. Sheet e Drawer

Diferença prática

Sheet

Abre pela lateral. Ótimo para:

- filtros;
- visualização rápida de detalhes;
- menu mobile;
- edição lateral.

Drawer

Abre em formato de gaveta, geralmente de baixo para cima. Ótimo para:

- mobile;
- ações rápidas;
- detalhes condensados.

Padrão recomendado

- Desktop: usar **Dialog** ou **Sheet**;
- Mobile: usar **Drawer** quando fizer sentido;
- Filtros laterais: **Sheet**.

Exemplo com Sheet

```
import { Button } from "@components/ui/button"
import {
  Sheet,
  SheetContent,
  SheetDescription,
  SheetHeader,
  SheetTitle,
  SheetTrigger,
} from "@components/ui/sheet"

export function FiltrosClientes() {
  return (
    <Sheet>
      <SheetTrigger asChild>
        <Button variant="outline">Filtros</Button>
      </SheetTrigger>

      <SheetContent>
        <SheetHeader>
          <SheetTitle>Filtrar clientes</SheetTitle>
          <SheetDescription>
            Refine a listagem por perfil e período.
          </SheetDescription>
        </SheetHeader>

        <div className="mt-6 space-y-4">
          {/* filtros */}
        </div>
      </SheetContent>
    </Sheet>
  )
}
```

5. Formulários

5.1. Bibliotecas recomendadas

O padrão adotado deve ser:

```
npm install react-hook-form zod @hookform/resolvers
```

5.2. Por que usar isso

- validação organizada;
- formulários menores e mais claros;
- erros por campo;
- schemas reutilizáveis;
- melhor manutenção.

5.3. Exemplo de schema Zod

```
import { z } from "zod"

export const clienteSchema = z.object({
  nome: z.string().min(3, "Informe o nome completo"),
  cpf: z.string().min(11, "CPF inválido"),
  telefone: z.string().min(10, "Telefone inválido"),
  email: z.string().email("E-mail inválido").optional().or(z.literal("")),
})
```

5.4. Exemplo com React Hook Form

"use client"

```
import { zodResolver } from "@hookform/resolvers/zod"
import { useForm } from "react-hook-form"
import { z } from "zod"
```

```
import { Button } from "@components/ui/button"
import {
  Form,
  FormControl,
  FormField,
  FormItem,
  FormLabel,
  FormMessage,
} from "@components/ui/form"
import { Input } from "@components/ui/input"
```

```
const formSchema = z.object({
  nome: z.string().min(3, "Informe o nome do cliente"),
  telefone: z.string().min(10, "Informe um telefone válido"),
})
```

```
export function FormCliente() {
  const form = useForm({
    resolver: zodResolver(formSchema),
    defaultValues: {
      nome: "",
      telefone: "",
    },
  })
}
```

```
function onSubmit(values) {
  console.log(values)
}
```

```
return (
  <Form {...form}>
    <form onSubmit={form.handleSubmit(onSubmit)} className="space-y-6">
      <FormField
        control={form.control}
        name="nome"
        render={({ field }) => (
          <FormItem>
            <FormLabel>Nome</FormLabel>
            <FormControl>
              <Input placeholder="Nome do cliente" {...field} />
            </FormControl>
            <FormMessage />
          </FormItem>
        )}
      />
```

```
    <FormField
      control={form.control}
      name="telefone"
      render={({ field }) => (
        <FormItem>
          <FormLabel>Telefone</FormLabel>
          <FormControl>
            <Input placeholder="(00) 00000-0000" {...field} />
          </FormControl>
          <FormMessage />
        </FormItem>
      )}
    />
```

```
<Button type="submit">Salvar cliente</Button>
</form>
</Form>
)
}
```

6. Inputs e campos de formulário

6.1. Input

Usar para:

- texto curto;
- CPF;
- telefone;
- número de OS;
- grau esférico/cilíndrico, se o campo for simples.

```
import { Input } from "@components/ui/input"
```

```
<Input placeholder="Buscar cliente..." />
```

6.2. Textarea

Usar para:

- observações da venda;
- anotações clínicas não sensíveis em contexto autorizado;
- instruções ao laboratório;
- descrição de problema.

```
import { Textarea } from "@components/ui/textarea"
```

```
<Textarea placeholder="Observações da ordem de serviço" />
```

6.3. Select

Usar para listas médias e estáveis:

- status da OS;
- vendedor;
- forma de pagamento;

- laboratório.

6.4. Checkbox

Usar para seleção múltipla:

- deseja receber mensagem;
- comprovante entregue;
- receita anexada;
- cliente autorizou contato.

6.5. Radio Group

Usar para escolha única com poucas opções:

- tipo de cliente: pessoa física / jurídica;
- tipo de lente;
- entrega: retirada / delivery.

6.6. Switch

Usar para estados binários rápidos:

- ativo / inativo;
- urgente / normal;
- permitir notificação;
- entregue / não entregue, se o contexto for muito direto.

7. Badge

Quando usar

Badge é um dos componentes mais importantes do sistema.

Exemplos de status

OS:

- Aguardando laboratório
- Em montagem
- Pronta para retirada
- Entregue
- Cancelada

Pagamento:

- Pago
- Parcial
- Pendente

Cliente:

- Novo
- Recorrente
- VIP

Exemplo

```
import { Badge } from "@components/ui/badge"

export function StatusOS() {
  return <Badge variant="secondary">Em produção</Badge>
}
```

Recomendação

Criar um componente próprio:

components/os/status-os-badge.jsx

Ele recebe o status e devolve o badge correto.

8. Dropdown Menu

Quando usar

Ações de linha em tabelas:

- visualizar;
- editar;
- alterar status;
- imprimir;
- excluir.

Exemplo conceitual

```
<DropdownMenu>
  <DropdownMenuTrigger asChild>
    <Button variant="ghost" size="icon">...</Button>
  </DropdownMenuTrigger>
  <DropdownMenuContent align="end">
```

```
<DropdownMenuItem>Ver detalhes</DropdownMenuItem>
<DropdownMenuItem>Editar</DropdownMenuItem>
<DropdownMenuItem>Imprimir</DropdownMenuItem>
<DropdownMenuItem className="text-destructive">Excluir</DropdownMenuItem>
</DropdownMenuContent>
</DropdownMenu>
```

9. Table e Data Table

9.1. Table simples

Usar para pequenas listagens ou detalhes fixos.

Exemplos:

- resumo de itens da venda;
- histórico compacto de alterações;
- resumo de parcelas.

9.2. Data Table com TanStack Table

Usar para listagens principais do sistema:

- clientes;
- OS;
- envelopes;
- pagamentos;
- usuários;
- laboratórios.

Recursos esperados

- filtro global;
- busca por coluna;
- ordenação;
- paginação;
- ações por linha;
- seleção em lote.

Dependência

npm install @tanstack/react-table

Organização recomendada

components/clientes/clientes-columns.jsx
components/clientes/clientes-data-table.jsx

10. Tabs

Quando usar

Telas com informações agrupadas no mesmo contexto.

Exemplo: página de cliente

Dados cadastrais | Envelopes | OS | Compras | Anexos | Observações

Exemplo: página de envelope

Resumo | Receita | Produtos | Pagamento | Histórico

11. Accordion e Collapsible

Accordion

Usar para blocos extensos em que uma seção por vez pode ganhar destaque.

Exemplos:

- detalhes da receita;
- medidas ópticas;
- condições comerciais;
- instruções ao laboratório.

Collapsible

Usar para mostrar/esconder um conteúdo simples.

Exemplos:

- detalhes adicionais;

- filtros avançados;
 - histórico secundário.
-

12. Calendar, Popover e filtros de data

Calendar

Usar para:

- data de nascimento;
- data da compra;
- prazo de entrega;
- período de relatórios.

Popover

Usar para encapsular o calendário, criando um Date Picker elegante.

Exemplo de uso esperado

[Selecionar período] -> abre popover com calendário

13. Tooltip

Quando usar

Explicações curtas em:

- ícones;
- siglas;
- colunas compactas;
- botões sem texto.

Exemplos:

- DNP;
 - Altura;
 - Status de malote;
 - Ícone de impressão.
-

14. Breadcrumb

Quando usar

Telas internas com navegação hierárquica.

Exemplos

Clientes > Maria Eduarda > Envelope #2048

OS > OS #421 > Histórico

Relatórios > Vendas por período

15. Skeleton

Quando usar

Sempre que houver carregamento de dados.

Exemplos

- cards do dashboard;
- tabela de clientes;
- detalhes da OS;
- histórico de envelope.

Nunca deixar a tela “pular” abruptamente enquanto carrega.

16. Pagination

Quando usar

- clientes;
- OS;
- histórico;
- relatórios tabulares.

Regra

Evitar scroll infinito em telas administrativas. Paginação é mais previsível e auditável.

17. Command

Quando usar

Buscar rapidamente:

- clientes;
- telas;
- OS;
- envelopes;
- ações do sistema.

Ideias para o Vision Manager

17.1. Busca global

Atalho como:

Ctrl + K

Permite procurar:

- cliente;
- número de OS;
- CPF;
- telefone;
- pedido.

17.2. Combobox pesquisável

Pode ser usado para:

- selecionar cliente em nova OS;
 - selecionar vendedor;
 - selecionar laboratório.
-

18. Sonner / Toast

Quando usar

Feedback curto após ações:

- cliente criado;
- OS atualizada;
- envelope salvo;
- erro no upload;
- pagamento registrado.

Exemplo

```
import { toast } from "sonner"
```

```
<Button onClick={() => toast.success("Cliente cadastrado com sucesso")}>Salvar</Button>
```

Instalação do Toaster

No layout raiz da aplicação, inserir o componente `Toaster`.

19. Chart

Quando usar

Dashboard e relatórios.

Bons gráficos para o sistema

- vendas por mês;
- ticket médio;
- OS por status;
- volume por vendedor;
- envelopes criados por período;
- taxa de entrega no prazo.

Regra

Gráfico é para revelar padrão, não para decorar dashboard. Se não ajuda a decidir, não entra.

20. Componentes extras que valem instalar depois

Mesmo não estando no comando inicial, eu recomendo considerar:

```
npx shadcn@latest add sidebar empty spinner input-group button-group
```

Por quê

Sidebar

Perfeita para o menu principal da plataforma.

Empty

Excelente para telas vazias:

- nenhum cliente encontrado;
- nenhuma OS cadastrada;
- nenhuma venda no período.

Spinner

Feedback rápido em botões de submit.

Input Group

Busca com ícone, prefixos e ações acopladas.

Button Group

Filtros agrupados, seleção de visão e ações secundárias.

21. Padrões visuais recomendados para o Vision Manager

21.1. Botões

- 1 botão principal por bloco ou cabeçalho;
- evitar 4 botões gritantes na mesma linha;

- ações secundárias em outline ou ghost.

21.2. Cards

- não transformar tudo em card;
- card serve para destacar grupos de informação;
- listagens extensas continuam em tabela.

21.3. Modais

- curtos;
- objetivos;
- foco em uma decisão ou formulário pequeno.

21.4. Tabelas

- ações no dropdown;
- filtros claros acima;
- badges de status;
- primeira coluna identificadora muito legível.

21.5. Formulários

- agrupar por assunto;
 - usar título de seção;
 - validar em tempo útil;
 - mensagens de erro humanas.
-

22. Sugestão de componentes de negócio a criar primeiro

Clientes

- `ClienteFilters`
- `ClienteStatusBadge`
- `ClienteResumoCard`
- `ClienteDrawerResumo`

Envelopes

- EnvelopeStatusBadge
- EnvelopeResumoCard
- EnvelopeTimeline
- EnvelopeFilters

OS

- OSStatusBadge
- OSTableActions
- OSFilters
- OSResumoCard

Dashboard

- KpiCard
 - GraficoVendasPeriodo
 - GraficoOSStatus
 - ResumoAtendimentosDia
-

23. Receitas de tela para acelerar o desenvolvimento

23.1. Tela de clientes

Componentes:

- Breadcrumb;
- PageHeader;
- Button “Novo cliente”;
- Input de busca;
- Sheet de filtros;
- Data Table;
- Pagination;
- Sonner.

23.2. Tela de envelope digital

Componentes:

- Breadcrumb;

- Card de resumo;
- Badge de status;
- Tabs;
- Accordion para dados extensos;
- Dropdown Menu de ações;
- Alert Dialog para exclusão/cancelamento.

23.3. Tela de OS

Componentes:

- Cards KPI pequenos;
- filtros em Sheet;
- tabela com badges;
- dropdown por linha;
- Dialog rápido para status;
- Sonner para confirmação.

23.4. Dashboard

Componentes:

- Cards de KPI;
- Charts;
- Skeleton;
- Date Picker;
- Select de período;
- Table resumida de OS recentes.

24. Exemplo de padrão de página

```
export default function ClientesPage() {
  return (
    <div className="space-y-6">
      <header className="flex flex-col gap-4 md:flex-row md:items-center md:justify-between">
        <div>
          <h1 className="text-2xl font-bold">Clientes</h1>
          <p className="text-sm text-muted-foreground">
            Gerencie cadastros, histórico e envelopes digitais.
          </p>
        </div>
        <Button>Novo Cliente</Button>
      </header>
```

```

<Card>
  <CardContent className="p-6">
    <div className="mb-4 flex flex-col gap-3 md:flex-row md:items-center
md:justify-between">
      <Input placeholder="Buscar por nome, CPF ou telefone" className="md:max-w-sm" />
      <Button variant="outline">Filtros</Button>
    </div>

    { /* DataTable aqui */ }
  </CardContent>
</Card>
</div>
)
}

```

25. Diretriz final

O projeto deve seguir esta lógica:

1. **shadcn/ui resolve o visual-base;**
2. **React Hook Form + Zod resolvem formulários;**
3. **TanStack Table resolve listagens complexas;**
4. **componentes próprios resolvem o domínio da ótica.**

Esse padrão dá um sistema:

- mais rápido de desenvolver;
- mais consistente;
- mais fácil de manter;
- mais profissional para crescer sem virar uma colcha de retalhos.